TP 3 : Méthodes de segmentation basées région

L'objectif de ce TP est d'implémenter l'algorithme de croissance de région vu en cours (cf. slides du cours). L'algorithme se divise en trois étapes :

1. Sélectionner des graines (avec la souris)
2. Définir le prédicat de similarité entre les régions
3. Grandir les régions, à partir des graines, qui vérifie le critère de similarité

Nous allons par le suite implémenter étape par étape pour la mise en œuvre cet algorithme.

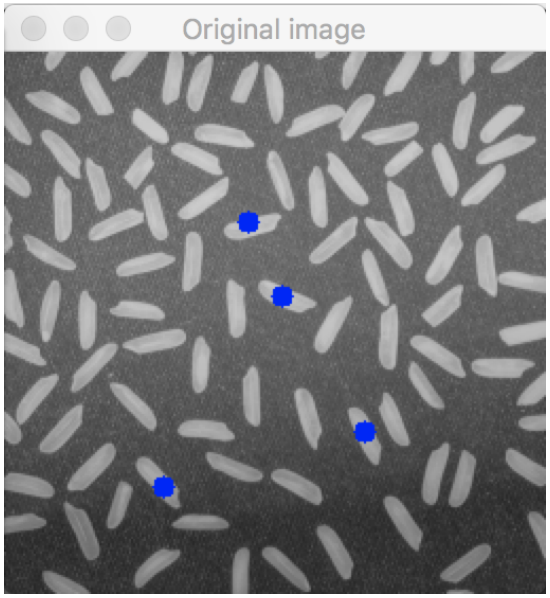
Les fichiers de supports du TP sont à récupérer sur Arche.

Attention : Vous ne modifiez pas le fichier `main.cpp` et seulement le fichier `fonctions.cpp` est à compléter.

Dépôt sur Arche : Vous rendez le fichier `fonctions.cpp`, `main_GrabCut.cpp` et un petit rapport en pdf contenant les résultats d'exécution de votre programme.

Exercice 1 _____ Fenêtre interactive pour les graines

Cette premier étape consiste à choisir les graines initiaux de l'algorithme avec les actions de cliquer sur le bouton de gauche de la souris. Pour chaque clic, on affichera un disque au point sélection comme illustré ci-dessous.



Cette étape est implémentée dans `main.cpp` via les fonctions :

— associer l'événement de la souris [1] sur une fenêtre :

```
void setMouseCallback(const string& winname, MouseCallback onMouse,...);
```

1. https://docs.opencv.org/4.8.0/d7/dfc/group__highgui.html#ga89e7806b0a616f6f1d502bd8c183ad3e

— Événement associé au bouton droit `EVENT_LBUTTONDOWN` [2] pour `MouseCallback`, eg. `void onMouse(int event, int x, int y, int flags, void* userdata);` Les coordonnées de la souris sont stockées dans les variable `x,y` du callback `onMouse`. Stocker tout les graines sélectionnées par l'utilisateur dans un vecteur de points, eg. `vector<Point> graines`

— Dessiner un disque [3] sur une image :
`void circle(Mat& img, Point center, int radius, const Scalar& color, int thickness=1, ...) ;`

Testez cette implémentation en compilant le code et l'exécutez. Les graines résultants sont stockés dans un vector des points.

Exercice 2 — Prédicats de similarité

Le prédicat de similarité permet de dire si un pixel appartient à un région définie par une graine Vous allez maintenant implémenter trois fonctions de prédicat différent pour tester leur efficacité.

Le premier prédicat est basé sur l'intensité des pixels de tel que, étant donné un point p, la graine g_i de la région R_i et un seuil, le prédicat $P_1(p, g_i)$ est défini par :

$$P_1(p, g_i) = \begin{cases} true \text{ si } |I(p) - I(g_i)| < seuil \\ false \text{ sinon} \end{cases} \quad (1)$$

Le deuxième prédicat est basé sur la différence d'intensité du point p par rapport à la moyenne des intensités des points dans R_i, le prédicat $P_2(p, R_i)$ est défini par :

$$P_2(p, R_i) = \begin{cases} true \text{ si } |I(p) - moyenne(R_i)| < seuil \\ false \text{ sinon} \end{cases} \quad (2)$$

Le troisième prédicat est basé sur la variance intra-region, le prédicat $P_3(p, R_i)$ est défini par :

$$P_3(p, R_i) = \begin{cases} true \text{ si } variance(R_i \cup p) < seuil \\ false \text{ sinon} \end{cases} \quad (3)$$

Implémentez ces trois prédicats dans `fonctions.cpp` :

```
bool predicat1(const Mat& src, Point g, Point p, int seuil) ;
bool predicat2(const Mat& src, const vector<Point>& Ri, Point p, int seuil) ;
bool predicat3(const Mat& src, const vector<Point>& Ri, Point p, int seuil) ;
```

Nous allons tester l'efficacité de ces différents prédicats avec l'implémentation de l'algorithme de croissance de région dans l'exercice suivant.

2. https://docs.opencv.org/4.8.0/d7/dfc/group__highgui.html
3. https://docs.opencv.org/4.8.0/d6/d6e/group__imgproc__draw.html#gaf10604b069374903dbd0f0488cb43670

Exercice 3 _____ Algorithme de croissance de région

Cette exercice consiste à implémenter l'algorithme de croissance de région (cf. slides du cours) comme suivant, avec I l'image entrée et I' image résultante de la segmentation.

```
label = 1
adjacent = 8
initialiser I' à 0
Pour chaque graine g_i de R_i, faire
    I'(g_i) = label
    S est une pile (vector de points) contient les voisins de  g_i
    Tant que S n'est pas vide
         p est 1er élément dans S
         Si I'(p) == 0 et P(p, g_i) == true alors
              I'(p) = label /* marquer p avec le label de R_i
                              ajouter les voisins de p dans S */
    label += 1 /* pour afficher l'image segmentée, pouvez prendre
            une échelle de 10 * label comme l'intensité des classes */
```

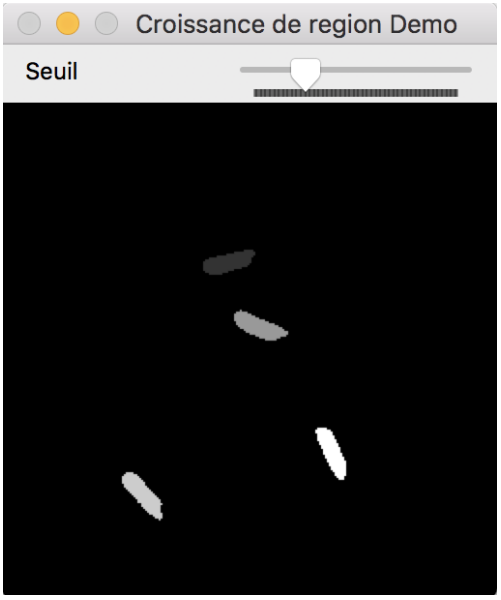
Implémentez cette fonction dans `fonctions.cpp` :

```
Mat croissanceRegion(const Mat& src, const vector<Point>& germes,
                     int seuil, int predicat) ;
```

Implémentez aussi la fonction qui calcule le voisinage d'un point dans `fonctions.cpp` :

```
vector<Point> voisinage(Point p, int adjacence) ;
```

Testez votre implémentation sur différentes images. Vous remarquerez que le seuil utilisé pour le prédicat est déterminé par un trackbar dans la fenêtre contenant le résultat de la segmentation. Voici un exemple de résultat obtenu avec le prédicat 1 :



Exercice 4 — Graph cut

Cette exercice a pour l'objectif de tester l'implémentation de l'algorithme de graph-cut[4] dans OpenCV de La méthode proposé dans le papier : "GrabCut" : interactive foreground extraction using iterated graph cuts, C. Rother, V. Kolmogorov and A. Blake, SIGGRAPH 2004. Il s'agit de la fonction suivante :

```
void grabCut (InputArray img, InputOutputArray mask, Rect rect,
        InputOutputArray bgdModel, InputOutputArray fgdModel,
        int iterCount, int mode = GC_EVAL);
avec
image : image entrée
mask  : masque indiqué le background
rect  : ROI contenant l'objet à segmenter, il est activé lors que
        mode=GC_INIT_WITH_RECT
bgdModel : tableau contenant le background
fgdModel : tableau contenant le foreground
iterCount : nombre d'intération maximum
mode :
  - GC_INIT_WITH_RECT : l'utilisation du paramètre rect comme le masque
  - GC_INIT_WITH_MASK : l'utilisation du paramètre mask comme le masque
  - GC_EVAL : l'algorithme devrait simplement reprendre.
```

Voici le résultat obtenu avec l'algorithme GrabCut sur `daisy.png` :



Image originale



Image résultante

Créer un fichier `main_GrabCut.cpp` pour tester la fonction `grabCut` ci-dessus.

4. https://www.docs.opencv.org/trunk/d7/d1b/group__imgproc__misc.html#ga909c1dda50efcbeaa3ce126be862b37f